

OPERATOR OVERLOADING

Operator Overloading

- The only built-in operations on classes are assignment and member selection
- Other operators cannot be applied directly to class objects
- C++ allows you to extend the definitions of most of the operators to work with classes
- This is called operator overloading
- That is, the `+` in `a+b` will add the variables if `a` and `b` are integers, but will call a different function (if overloaded) if `a` and `b` are variables of a user defined type.

Operator Overloading (continued)

- Type of Polymorphism
 - Providing implementation of an already existing operator for user defined types.
- Can overload most C++ operators
- Cannot create new operators
- Most existing operators can be overloaded to manipulate class objects
- Write an operator function to overload an operator

Operator Overloading: Rules

- You can not change numbers of operands. A binary operator (for example `+`) must always take two operands.
- You can not change the precedence of the operators.
 - `*` comes always before `+`
- Everything you can do with an overloaded operator you can also do with a function. However, by making your listing more intuitive, overloaded operators make your programs easier to write, read, and maintain.
- Operator overloading is mostly used with objects. We will discuss this topic later more in detail.

Syntax

```
return-type class-name :: operator op (argList)
{
    //function body
}
```

OR

```
return-type operator op (argList)
{
    //function body
}
```

- Where **return-type** is the type of value returned by the specific operation.
- **op** is the operator being overloaded.
- **operator op** is the function name where, where operator is a keyword.

What can't be overloaded?

- You can't overload operators that don't already exist in C++. You can overload only the built-in operators.
- You cannot overload the following operators:
 - Class member access operator (**.** (**dot**))
 - Pointer to members (**.*** (**dot-asterisk**))
 - Scope resolution operator (**::**)
 - Conditional Operator (**?:**)
 - Size Operator (**sizeof**)

Operators that can be overloaded

+	-	*	/	%	^
&		~	!	,	=
<	>	<=	>=	++	--
<<	>>	==	!=	&&	
+=	-=	/=	%=	^=	&=
=	*=	<<=	>>=	[]	()
->	->*	new	new []	delete	delete []

Overloading the + operator

```
/* A class to define complex numbers */
class TComplex
{
    float real,img;
public:
    TComplex(){ real = 0; img = 0;}
    TComplex(float r, float i): real(r), img(i) {}
    TComplex operator + (TComplex&); // header of operator+ function
};

/* The Body of the function for operator + */
TComplex TComplex::operator + (TComplex& z)
{
    TComplex result;
    result.real = real + z.real;
    result.img = img + z.img;
    return result;
}
```

```
int main()
{
    TComplex z1(3.2, 5.5);
    TComplex z2(1.5, 7.0), z3;
    z3 = z1 + z2; //operator +
    return 0;
}
```


Overloading the assignment operator (=)

```
void TComplex::operator=(const ComplexT& z)
{
    real = z.real;
    img = z.img;
}
```

- You don't need to write such an assignment operator function, because the operator provided by the compiler does the same thing.
- In general, you don't want to let the compiler do this for you.
- With classes of any sophistication (especially if they contain pointers!) you want to explicitly create an operator =.

Overloading the assignment operator (=)

IntArray example (assign_op.cpp)

```
void IntArray::operator = (const IntArray &s)
{
    size = s.size;
    contents = new int[size];
    //copies contents of one array to another
    for (int i = 0; i < size; i++)
        contents[i] = s.contents[i];
}
```

Return value of the assignment operator (=)

- When there's a void return value, you can't chain the assignment operator (as in `a = b = c`).
- To fix this, the assignment operator must return a reference to the object that called the operator function (its address).

```
IntArray& IntArray::operator = (const IntArray &s)
{
    cout<<"Overloaded Operator"<<endl;
    size = s.size;
    contents = new int[size];
    //copies contents of one array to another
    for (int i = 0; i < size; i++)
        contents[i] = s.contents[i];

    return *this;
} //IntArray example (assign_op.cpp)
```

Overloading Unary Operators

- Unary operators operate on a single operand. Examples are the increment (++) and decrement (--) operators; the unary minus, as in -5; and the logical not (!) operator.
- Unary operators take no arguments, they operate on the object for which they were called. Normally, this operator appears on the left side of the object, as in !obj, -obj, and ++obj.
- Overloading the unary ++ operator (**unary.cpp**).

Overloading the “[]” operator

- Same rules apply to all operators. So we don't need to discuss each operator. However, we will examine some interesting operators.
- One of the interesting operators is the subscript operator.
- It can be declared in two different ways:

```
return-type& operator [ ](paramtype)  
{ }
```

OR

```
const returntype& operator [ ] (paramtype) const { }
```

Overloading the “[]” operator

- The first declaration can be used when the overloaded subscript operator modifies the object. The second declaration is used with a const object; in this case, the overloaded subscript operator can access but not modify the object.
- SafeArray Example (**safe_array_func.cpp** and **safe_array.cpp**)

Operator Functions as Member Functions and Nonmember Functions

- Most of the operators can be overloaded either as member or nonmember functions.
- To make an operator function be a member or nonmember function of a class, keep the following in mind:
 - The function that overloads any of the operators `()`, `[]`, `->`, or `=` for a class must be declared as a member of the class.
- Example (**Rect.cpp**)

Stream insertion (<<) and Stream extraction (>>) operators

- Explanation with example (**StreamOperators.cpp**)

THANKS
